

[4]Sprawozdanie Zadanie IV

Wydział i kierunek:	WIiT, Bioinformatyka
Imię i nazwisko prowadzącego: Prof. Dr. hab. Inż. Marta Kasprzak	Data oddania sprawozdania:
Imię i nazwisko: Kacper Gałaj 163827	Ocena:

1. Algorytm

Program rozwiązuje problem odtwarzania punktów na odcinku na podstawie listy odległości. Najpierw wczytuje dane, sprawdza czy są sensowne, wyznacza L jako największą odległość i startuje od punktów 0 oraz L . Potem **rekurencyjnie** wybiera największą pozostałą odległość y i próbuje wstawić nowy punkt w pozycji y albo $L - y$. Jeśli po takim wstawieniu nie da się usunąć wszystkich wymaganych odległości z multizbioru, to cofa zmiany i sprawdza drugą opcję. Działa to jak klasyczny backtracking z dwiema możliwościami na każdym kroku.

Uwaga

Z uwagi na brak wyników w czasach porównywalnych (dla każdej instancji >1 ms) stworzyłem algorytm brute force'owy, który porównuje wszystkie możliwe ścieżki i wycieczki i wybiera najlepszy.

2. Pseudokod

```
SOLVE(d, points, L):
```

```
    jeśli d jest puste:
```

```
        return true
```

```
    candidate_points = wszystkie klucze z d posortowane malejąco
```

```

for p in candidate_points:

    jeśli p już jest w points:
        continue

    (ok, removed) = TAKE_DISTANCES(p, points, d)

    jeśli ok:
        dodaj p do points
        jeśli SOLVE(d, points, L) == true:
            return true

    usuń p z points
    RESTORE(d, removed)

return false

```

▼ [1] Test: Instancja referencyjna(ASC)

Plik `ins-PDP-12a-asc.txt`

```

2 3 4 4 5 5 6 6 6 7 7 8 8 9 10 11 11 11 11 12 13 13 14 14 15 15 16 16 17 18 19 19
19 20 20 21 21 22 23 23 25 25 25 26 26 27 27 27 30 30 31 31 32 32 35 35 3
6 36 38 38 38 40 41 42 42 43 44 46 46 49 51 52 53 57 57 61 63 67

```

Wynik dla kodu który ma funkcje przycinania złych gałęzi:

```
./exe4 ./data/example1-asc.txt
Czas: 0 ms
Długosc: 67
Mapa: 6 8 7 4 2 5 9 3 8 5 6 4
Zgodnosc mapy: TAK
Liczba odwiedzonych wezłow: 18
Liczba sprawdzonych kandydatow: 29
```

Wynik dla kodu który robi bruteforce(wczytuje wszystkie możliwości:

```
./exe4brut ./data/example1-asc.txt
Czas: 816 ms
Długosc: 67
Mapa: 6 8 7 4 2 5 9 3 8 5 6 4
Zgodnosc mapy: TAK
Liczba odwiedzonych wezłow: 13529
Liczba sprawdzonych kandydatow: 382339
```

Wnioski:

Ze względu na zaawansowanie technologii w 2026, możemy zauważyć różnice że program który wybiera ścieżkę nie testując wszystkich kandydatów początku(**bruteforce**), ma czas **>1ms**. Z kolei algorytm biorący wszystkie sekwencje na siłę tak długo aż nie znajdzie pierwszej, trwa aż* **816ms**.

Dlaczego bruteforce ma tak o tyle dłuższe wyniki?

- Brute force rozgalezia sie maksymalnie: w kazdym kroku testuje wszystkie mozliwe kandydaty z multizbioru odleglosci, wiec liczba prob i wezlow rosnie lawinowo (tu ~750x wiecej wezlow i ~13 tys. razy wiecej kandydatow).

- Obie wersje mają to samo przycinanie, ale przy mniejszym rozgalezieniu przycinanie w exe4.cpp szybko kończy szukanie, stąd czas < 1 ms.

▼ [2] TEST: Lustrzane motywy

example4-asc.txt

```
12 13 15 25 25 27 35 38 42 47 48 54 57 57 60 63 66 66 74 79 79 82 82 89
93 95 101 104 107 108 112 114 120 123 135 136 136 137 139 146 150 151 161 1
61 164 167 171 184 186 193 194 196 199 199 205 209 211 212 221 224 228 23
0 239 241 247 250 253 259 262 265 272 273 286 287 287 294 300 307 3
07 313 319 321 329 334 344 346 351 360 366 373 376 395 398 400 408 4
08 420 423 423 433 433 446 458 458 471 474 480 483 512 512 515 528 5
37 540 559 594 594 607 619 673
```

example4-desc.txt

```
673 619 607 594 594 559 540 537 528 515 512 512 483 480 474 471 458
458 446 433 433 423 423 420 408 408 400 398 395 376 373 366 360 35
1 346 344 334 329 321 319 313 307 307 300 294 287 287 286 273 272 26
5 262 259 253 250 247 241 239 230 228 224 221 212 211 209 205 199 199
196 194 193 186 184 171 167 164 161 161 151 150 146 139 137 136 136 135 123
120 114 112 108 107 104 101 95 93 89 82 82 79 79 74 66 66 63 60 57 57 54
48 47 42 38 35 27 25 25 15 13 12
```

wyniki dla obu plików:

```
./exe4 ./data/example4-asc.txt
Czas: 0 ms
Długość: 673
Mapa: 79 57 25 38 74 27 66 42 15 89 47 35 13 12 54
Zgodność mapy: TAK
Liczba odwiedzonych węzłów: 15
Liczba sprawdzonych kandydatów: 20
```

```
./exe4 ./data/example4-desc.txt
Czas: 0 ms
Dlugosc: 673
Mapa: 79 57 25 38 74 27 66 42 15 89 47 35 13 12 54
Zgodnosc mapy: TAK
Liczba odwiedzonych wezlow: 15
Liczba sprawdzonych kandydatow: 20
```

Wnioski:

W obydwu rezultatach mimo odwróconych kolejności wynik jest taki sam, w algorytmie zastosowałem zliczanie w `map<int,int> (Counts)`, `map` w C++ ma to do siebie, że automatycznie trzyma klucze w porządku rosnącym niezależnie od instancji wejściowej.

[3] TEST: "Długa próba"

`ins-PDP-14a-asc.txt`

```
2 3 3 4 4 5 5 5 6 6 6 6 7 7 8 8 8 9 9 10 11 11 11 11 12 12 13 13 14 14 14 15 15 15
16 16 17 18 19 19 19 20 20 20 20 21 21 22 23 23 23 25 25 25 26 26 27 27 27
27 28 30 30 30 31 31 31 32 32 33 34 35 35 35 36 36 36 38 38 38 38 39 40
41 41 41 42 42 43 44 46 46 46 47 49 50 50 51 52 53 53 55 57 57 58 58 61
61 63 63 66 66 67 69 71 72 73 76 77 81
```

```
./exe4 ./data/example11-asc.txt
Czas: 0 ms
Dlugosc: 81
Mapa: 5 3 6 6 8 7 4 2 5 9 3 8 5 6 4
Zgodnosc mapy: TAK
Liczba odwiedzonych wezlow: 15
Liczba sprawdzonych kandydatow: 21
```

```
./exe4brut ./data/example11-asc.txt
```

```
Czas: 880627 ms
```

```
Długosc: 81
```

```
Mapa: 5 3 6 6 8 7 4 2 5 9 3 8 5 6 4
```

```
Zgodnosc mapy: TAK
```

```
Liczba odwiedzonych wezłow: 11775591
```

```
Liczba sprawdzonych kandydatow: 463495775
```

Wnioski

- exe4.cpp ogranicza rozgalezienie do dwóch opcji (y oraz L-y) i szybko odcina złe galezie, dlatego drzewo przeszukiwania ma tylko 15 wezłow i 21 kandydatow.
 - exe4brut.cpp na kazdym poziomie rozwaza wszystkie mozliwe odleglosci jako kandydatow punktu, wiec rozgalezienie jest wielokrotnie wieksze, a liczba prob rosnie wykladniczo.
 - Dla tej instancji (120 odleglosci, czyli $k=14$ ciecia) nawet niewielki wzrost k powoduje rozrost liczby mozliwych gałęzi; brute force wiekszosc czasu spedza na slepych sciezках.
 - Dłuższy czas metody brute force wynika z przeszukiwania pełnej przestrzeni rozwiązań.
-